SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA14 **COURSE NAME: COMPILER DESIGN**

COURSE OUTCOME COVERED

CO2: *Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation (BL3)*

Assessment Tool Weightage: 20%

Duration :60 Mins
On Class
QUESTIONS: 25 **Total Marks:50**

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I Karanya S (192421309) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Karanya S

Annexure A

SHORT ANSWER TEST

Q1. A lexical analyzer produces the following token stream:
id + id
Explain how the parser validates the syntactic structure of the expression.

Q2. Given the grammar:
$S \rightarrow aS \mid b$
Determine whether the string aaab belongs to the language using leftmost derivation.

Q3. Remove left recursion from the grammar:
$A \rightarrow Aa \mid b$

Q4. Find FIRST(S) for the grammar:
$S \rightarrow aA \mid b$
$A \rightarrow c \mid \varepsilon$

Q5. Determine whether the grammar is suitable for top-down parsing.
$S \rightarrow Sa \mid b$
Justify your answer.

Q6. Apply left factoring to the grammar:
S → if E then S
S → if E then S else S

Q7. Construct a parse tree for the string:
a + b
using the grammar:
$E \rightarrow E + E$
$E \rightarrow id$

Q8. Write the recursive descent procedures for the grammar:
$S \rightarrow aA$
$A \rightarrow b$

Q9. Given the grammar:
$S \rightarrow AB$
$A \rightarrow a$
$B \rightarrow b$
Construct the predictive parsing table.

Q10. Demonstrate the first three actions of a shift-reduce parser for the input:
id + id

Q11. Identify the handle in the string:
id + id
for the grammar:
$E \rightarrow E + E$
$E \rightarrow id$

Q12. Construct the operator precedence relations for the operators:

+
*
where * has higher precedence than +.

Q13. Explain how operator precedence parsing handles the expression:
id * id

Q14. What is an LR parser? Illustrate the parsing process using the grammar:
S → aA
A → b

Q15. Given the grammar:
S → aA
A → b
Write the steps involved in constructing an SLR parsing table.

Q16. Compute FOLLOW(A) for the grammar:
S → AB
A → a
B → b

Q17. Construct the augmented grammar for:
S → a

Q18. Find the closure of the LR(0) item:
S' → •S
for the grammar:
S → a

Q19. Check whether the grammar is ambiguous:
E → E + E
E → id
using the string:
id + id + id

Q20. Construct a leftmost derivation for the string:
id + id
using the grammar:
E → E + E
E → id

Q21. Apply left factoring to the grammar:
S → aA
S → aB
A → b
B → c

Q22. Find FIRST(A) for the grammar:
A → x
A → y

Q23. Determine whether the grammar is suitable for predictive parsing.
S → aA | aB
A → b
B → c

Justify your answer.

Q24. Construct recursive descent parser procedures for the grammar:
$S \rightarrow aS$
$S \rightarrow b$

Q25. Build the predictive parsing table for the grammar:
$S \rightarrow aS$
$S \rightarrow b$
Show the table entries for the terminals:
a, b, $

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

/1/2026

CSA1403 – Compiler Design

Karanya . S
192421309

CO2 – Assessment Tool – 2 – Class Test

① Lexical analyzer token stream : id + id

The parser check whether the token sequence follows the grammar rules for an expression.

Given : id + id

id → expression

+ → operator

(8) id → another expression

Thus, id + id → gives syntactic structure.

② Leftmost derivation for aa ab

Grammar : $S \to aS \mid b$

Leftmost derivation

$S \to aS$
$\to aaS$
$\to aaaS$
$\to aaab$

∴ aaab belongs to the language.

3
P
S
3

③ Remove left recursion

$A \to Aa \mid b$

$A \to bA'$

$A' \to aA' \mid \varepsilon$

④ FIRST (S)

Grammar: $S \to aA \mid b$

$A \to c \mid \varepsilon$

Since, S starts with a or b.

$FIRST(S) = \{a, b\}$

⑤ Top-down parsing suitability

Grammar: $S \to Sa \mid b$

Recursion:

$S \to Sa$

⑥ Left factoring:

Given:

$S \to$ if E then S

$S \to$ if E then S else S

Common prefix: if E then S

Left-factored grammar:

$S \to$ if E then SS'

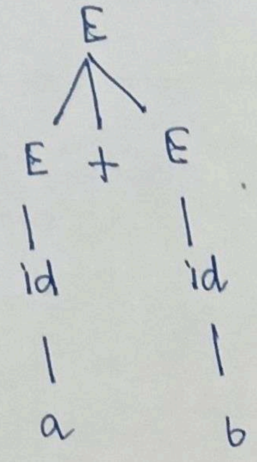
$S' \to$ else $S \mid \varepsilon$

(7) Parse tree for a+b

$E \rightarrow E + E$

$E \rightarrow id$

a and b



(8) Recursive descent

Grammar :

$S \rightarrow aA$

$A \rightarrow b$

```
procedure S ()
{
    match ('a');
    A();
}
procedure A()
{
    match('b');
}
```

(9) Predictive parsing table

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

Non-terminal	a	b	$
S	$S \rightarrow AB$	–	–
A	$A \rightarrow a$	–	–
B	–	$B \rightarrow b$	–

(10) First three shift - reduce actions

Input : id + id $

Initial stack : $

Stack	Input	Action
$id	+ id $	Shift id
$E	+ id $	Reduce $id \rightarrow E$
$E +	id $	Shift +

(11) Handle in id + id

$E \rightarrow E + E$

$E \rightarrow id$

The first handle — id

Reduced :

$E \rightarrow id$

(12) Operator precedance relations.

$+, * \rightarrow *$ has higher precedance

$+ < *$ and $* > +$

$+ = +$ / $+ > +$ depending on the precedence table convention.

(13) Operator precedence parsing : id * id

Shift first id.

Shifts *

Shifts the second id.

Reduces id

Reduces $E * E$ to E.

Thus, $id * id \rightarrow E * E \rightarrow E$

(14) What is an LR parser?

An LR parser is a bottom-up parser that scans input from Left to Right produces Rightmost derivation in reverse

Grammar:

$S \rightarrow aA$

$A \rightarrow b$

Parsing:

Stack	Action
\$	Shift a
\$a	Shift b
\$ab	Reduce $b \rightarrow A$
\$aA	Reduce $aA \rightarrow S$
\$S	Accept

15) Steps for constructing an SLR parsing table

For, $S \to aA$

$A \to b$

Steps: 1) Augment the grammar $S' \to S$

2) Construct the LR(0) item sets

3) Compute closure and GOTO for each item set

4) Construct the ~~FDA~~ ~~FDA~~ DFA of LR(0) items.

5) Compute FOLLOW sets

6) Fill the ACTION table using shift /reduce rules.

7) Fill the GOTO table for non-terminal

8) Check for shift-reduce or reduce-reduce conflicts.

(16) FOLLOW(A) $S \to AB$

$A \to a$

$B \to b$

Since A followed by B:

$$FOLLOW(A) = FIRST(B)$$

$$FIRST(B) = \{b\}$$

$$\therefore \quad FOLLOW(A) = \{b\}$$

(17) Augmented grammar: $S \to a$

$S' \to S$

$S \to a$

(18) Closure of $S' \to \cdot S$

Grammar

$S' \to S$
$S \to a$

Initial item:

$S' \to \cdot S$

Closure:

$S' \to \cdot S$

$S \to \cdot a$

(19) Grammar Ambigous ?

$E \to E+E$
$E \to id$

String:
$id + id + id$

Two diffrent parse:

$(E+E)+E$

and

$E+(E+E)$

(20) Leftmost derivation for id+ id

$E \to E+E$
$E \to id$

Derivation $E \Rightarrow E+E$
$\to id+E$
$id+id$

$\therefore E \to E+E \to id+E$
$\Rightarrow id+id$

Left Factoring:

Given $S \rightarrow aA$

$S \rightarrow aB$ common prefix is a.

Left - factored grammar

$S \rightarrow aS'$

$S' \rightarrow A|B$

With : $A \rightarrow b$

$B \rightarrow c$

(22) FIRST(A)

Grammar: $A \rightarrow x$

$A \rightarrow y$

$FIRST(A) = \{x, y\}$

(23) Predictive parsing suitability

Grammar:

$S \rightarrow aA \mid aB$

$A \rightarrow b$

$B \rightarrow c$

$FIRST(aA) = \{a\}$

$FIRST(aB) = \{a\}$

It must be left - factored:

$S \rightarrow aS'$

$S' \rightarrow A|B$

(24) Recursive decent parser:

$S \rightarrow aS \mid b$

Procedure:

```
procedure S()
{
    if (look ahead == 'a')
    {
        match('a');
        S();
    }
    else if (look ahead == 'b')
    {
        match('b');
    }
    else
        error();
}
```

(25) Predective parsing table.

$S \rightarrow aS$ FIRST $(aS) = \{a\}$

$S \rightarrow b$ FIRST $(b) = \{b\}$

Non-terminal	a	b	$
S	$S \rightarrow aS$	$S \rightarrow b$	-